

AV-Programming / Media Programming — Prüfungsvorbereitung

Test 1 am 2.6.2026 — 20 Min., keine Unterlagen, Taschenrechner erlaubt, Taxonomiestufe: Anwenden

RUNDE 1

Frage 1 — Dateigröße + Samplerate

Frage:

Du nimmst ein Sprachsignal auf: Dauer 3s, Samplerate 16.000 Hz, Mono, 16 Bit.

- a) Wie viele Bytes belegt die WAV-Datei (ohne Header)?
- b) Du änderst die Samplerate auf 8.000 Hz. Was ändert sich an der Qualität — und warum?

Deine Antwort:

- a) $3 \times 16k \times 1 \times 16/2$
- b) Geringe Qualität weil wenig Samplerate → höher Quantifizierungsfehler → Stutter

Feedback:

- a) ⚠ Formel richtig aber Ergebnis fehlt + Fehler: muss $\times (16/8)$ sein, nicht $\times (16/2)$.
Korrekt: $3 \times 16.000 \times 1 \times 2 = \text{**96.000 Bytes**} \approx 93,75 \text{ KB}$
- b) ✗ **Wichtige Verwechslung:**
 - **Quantisierungsfehler** hängt von der **Bittiefe** ab (nicht Samplerate)
 - **Samplerate** betrifft den **darstellbaren Frequenzbereich**
 - Korrekte Antwort: Nyquist → max. Frequenz = $8.000/2 = \text{4.000 Hz}$. Hohe Frequenzen (Zischlaute "s", "f") gehen verloren → Sprache klingt **dumpf**, nicht stutternnd.

Frage 2 — OpenCV Code ergänzen

Frage:

```
cap = cv.VideoCapture("video.mp4")
fps = cap.get(cv.CAP_PROP_FPS)
frame_count = cap.get(cv.CAP_PROP_FRAME_COUNT)
dauer = ____
```

Und: Was gibt `frame.shape` zurück bei einem Farbbild 1280×720?

Deine Antwort: `frame_count / fps` (shape nicht beantwortet)

Feedback:

- ✓ `frame_count / fps` korrekt
- `frame.shape = (720, 1280, 3)` — Reihenfolge: **(Höhe, Breite, Kanäle)**. Nicht Breite×Höhe! Kanäle = 3 weil BGR.

Frage 3 — BGR-Falle

Frage:

```
img = np.zeros((200, 200, 3), dtype=np.uint8)
img[:, :, 0] = 255 # <- soll rot sein
```

Was ist der Fehler?

Deine Antwort: Kein Fehler gesehen.

Feedback:

✗ **BGR-Falle!** OpenCV-Kanalreihenfolge: **B=0, G=1, R=2**

`img[:, :, 0] = 255` setzt **Blau**, nicht Rot.

Korrektur:

```
img[:, :, 2] = 255 # Kanal 2 = ROT ✓
```

Merke: `blue[:, :, 0]` , `green[:, :, 1]` , `red[:, :, 2]`

Frage 4 — Formatwahl WAV

Frage: Du analysierst Umgebungsgeräusche in Python — Pegelverlauf und spektrale Merkmale berechnen. Format + zwei Argumente + ein Nachteil?

Deine Antwort: WAV — kann Rohdaten erhalten → notwendig für Analyse. Nachteil: Datei zu groß.

Feedback:

- Format ✓
- Argument 1 ✓ Verlustfrei → keine Artefakte bei Merkmalsextraktion
- Argument 2 fehlte: **Direkter NumPy-Zugriff** ohne Dekodierungsschritt (`scipy.io.wavfile` , `librosa` lesen WAV nativ als Array)
- Nachteil ✓ Große Dateigröße (kein Kompressionsvorteil gegenüber FLAC)

Frage 5 — YCbCr Koeffizienten

Frage:

$$\begin{aligned} Y &= \text{___} * r + \text{___} * g + \text{___} * b \\ Cb &= (b - Y) * 0.564 \\ Cr &= (r - Y) * 0.713 \end{aligned}$$

Und: Warum hat Grün das höchste Gewicht?

Deine Antwort: 0.299, 0.587, 0.114 — Warum nicht gewusst.

Feedback:

- ✓ Koeffizienten korrekt
- Warum Grün: Das menschliche Auge ist **am empfindlichsten für grünes Licht**. Grün trägt am meisten zur wahrgenommenen Helligkeit bei → größter Anteil in der Luminanz-Berechnung. Deshalb auch 2 grüne Pixel in der **Bayer-Matrix**.

Frage 6 — Perlin vs. White Noise

Frage: Unterschied White Noise vs. Perlin Noise? Wofür braucht man die Fade-Funktion

$$f(t) = 6t^5 - 15t^4 + 10t^3 ?$$

Deine Antwort: White Noise = jeder Pixel zufälliger Wert 0–255. Perlin nicht gewusst.

Feedback:

- White Noise ✓
- Perlin Noise: **Kohärentes Rauschen** — benachbarte Pixel mathematisch verwandt → glatte, natürliche Übergänge (Wolken, Terrain). Basiert auf zufälligen Gradientenvektoren auf Gitter + Interpolation.
- Fade-Funktion: Glättet die Interpolation — ohne sie entstehen sichtbare Kanten an Gittergrenzen. Ableitung = 0 an den Enden → weiche Übergänge.

Frage 7 — JPEG Reihenfolge

Frage: Richtige Reihenfolge von A) Quantisierung, B) Chroma Subsampling, C) RGB→YCbCr, D) Huffman+RLE, E) DCT?

Deine Antwort: A → B → C → E → D (falsch)

Feedback:

X Richtig: **C → B → E → A → D**

- | | |
|-----------------------|--------------------------------------|
| C) RGB → YCbCr | (Farbraum trennen) |
| B) Chroma Subsampling | (Farbauflösung reduzieren) |
| E) DCT auf 8x8 Blöcke | (Raum → Frequenzbereich) |
| A) Quantisierung | ← EINZIGER verlustbehafteter Schritt |
| D) RLE + Huffman | ← verlustfrei, LETZTER Schritt |

Merkhilfe: **C**heck **B**ig **E**lephants **A**re **D**one

Frage 8 — NumPy Code schreiben

Frage: Schreibe `applyNegative(arr)`.

Deine Antwort:

```
for pixel in arr:  
    pixel = 255 - pixel
```

Feedback:

⚠ Idee richtig, aber zwei Probleme:

1. `pixel = ...` überschreibt nur lokale Variable — Original unverändert
2. For-Schleifen über NumPy sind langsam und unüblich

Korrekt (vektorisiert):

```
def applyNegative(arr):  
    return (255 - arr).astype(np.uint8)
```

Frage 9 — Container vs. Codec

Frage: Was ist der Unterschied zwischen Container und Codec?

Deine Antwort: Container organisiert Datei. Codec entscheidet wie wird Datei komprimiert.

Feedback:

✓ Korrekt. Präziser:

- **Container:** legt fest wie Daten, Metadaten und Streams in der Datei abgelegt werden. Beispiel: WAV, MP4, Ogg
- **Codec:** legt fest wie Audiodaten codiert/komprimiert sind. Beispiel: PCM, AAC, FLAC

```
musik.wav    = Container WAV  + Codec PCM  
podcast.m4a = Container MP4  + Codec AAC  
musik.ogg    = Container Ogg  + Codec Vorbis
```

Frage 10 — Mandelbrot

Frage:

```

for i in range(max_iteration):
    if __:
        z = z**2 + c
    else:
        break
image[y, x] = i

```

Was kommt in die Lücke? Was bedeutet `i` ?

Deine Antwort: $\text{abs}(z) < 2$ — `i` = schwarz

Feedback:

- Bedingung ✓ ($\text{abs}(z) \leq 2$)
- `i` = **Anzahl Iterationen bis** $|z| > 2$ (Escape-Geschwindigkeit)
 - Punkte innerhalb der Menge → `max_iteration` → **schwarz** ✓
 - Punkte außerhalb → verschiedene `i`-Werte → Farbe via Colormap (`cmap='inferno'`)

RUNDE 2

Frage 1 — Dateigröße Stereo

Frage: Dauer 10s, Samplerate 44.100 Hz, Stereo, 16 Bit.

- a) Bytes ohne Header?
- b) Bittiefe von 16 auf 8 Bit — was verschlechtert sich?

Deine Antwort:

- a) $10 \times 44,1\text{k} \times 2 \times 16/2$
- b) Höher Quantifizierungsfehler → Stutter

Feedback:

- a) ✗ $16/2$ → muss $16/8$ sein (Bits → Bytes = ÷8)
Korrekt: $10 \times 44.100 \times 2 \times 2 = \text{**1.764.000 Bytes**} \approx 1,68 \text{ MB}$

- b) ⚠️ Quantisierungsfehler wird größer ✓ aber Stutter falsch.

8 Bit = 256 Amplitudenstufen → Dynamikumfang: $6 \text{ dB} \times 8 = 48 \text{ dB}$ (statt 96 dB bei 16 Bit). Man hört: **Rauschen/Körnung**, besonders bei leisen Passagen.

Frage 2 — cvtColor + gray.shape

Frage:

```
gray = cv.cvtColor(frame, ____)
```

Und: `frame.shape = (720, 1280, 3)` → was ist `gray.shape` ?

Deine Antwort: (720, 1280) — Lücke nicht gewusst

Feedback:

- `gray.shape` ✓ (720, 1280) — kein 3. Kanal bei Graustufen
- Lücke: `cv.COLOR_BGR2GRAY`

Schema: `cv.COLOR_VON2NACH` . BGR zuerst weil OpenCV standardmäßig BGR liest.

Frage 3 — BGR-Falle

Frage:

```
img = np.zeros((100, 100, 3), dtype=np.uint8)
img[:, :, 2] = 255
```

Welche Farbe?

Deine Antwort: Red

Feedback: ✓ Kanal 2 = R in BGR → Rot.

Frage 4 — Formatwahl FLAC (Archivierung)

Frage: 90-min Interview für wissenschaftliche Analyse in 10 Jahren. Format + zwei Argumente + Nachteil?

Deine Antwort: FLAC — verlustfrei wie WAV aber weniger Speicher → gut für Archivierung. Nachteil: noch zu groß.

Feedback:

- Format ✓ (Schreibweise: FLAC, nicht FLAAC)
- Argument 1 ✓ Verlustfrei + 40–60% kleiner als WAV
- Argument 2 zu unspezifisch → besser: **Offenes, lizenzfreies Format** (in 10 Jahren noch lesbar) oder **Metadaten-Unterstützung** (Vorbis Comments: Datum, Sprecher direkt in Datei)
- Nachteil X "Noch zu groß" passt nicht zur Archivierung → besser: **Weniger universell** als WAV — nicht alle älteren Geräte/Programme können FLAC öffnen.

Frage 5 — YCbCr berechnen

Frage: Berechne Y für $R=255$, $G=0$, $B=0$. Warum wird Cb/Cr stärker komprimiert als Y?

Deine Antwort: $Y = 0.299R + 0.578G + 0.141B$ (Koeffizienten leicht falsch). Begründung: Man sieht Grün mehr als Helligkeit.

Feedback:

- ⚠ Koeffizienten: **0.587** (nicht 0.578) und **0.114** (nicht 0.141) — auswendig lernen!
- Ergebnis: $Y = 0.299 \times 255 = **76.2 \approx 76**$
- Begründung X → korrekt: Das Auge ist **empfindlicher für Helligkeitsunterschiede (Y) als für Farbunterschiede (Cb/Cr)**. Wir bemerken kaum fehlende Farbdetails → Chroma Subsampling 4:2:0 ohne sichtbaren Qualitätsverlust möglich.

Frage 6 — Perlin Fade-Funktion

Frage:


```
def fade(t):  
    return ____
```

Was berechnet `result = top + y_fade * (bottom - top)` ?

Deine Antwort: `10*t**5` ... — Interpolation allgemein

Feedback:

- Vollständige Fade-Funktion: `return 6*t**5 - 15*t**4 + 10*t**3` — Reihenfolge: **6, -15, 10**
- Letzte Zeile ✓: **Lineare Interpolation (lerp) in y-Richtung** zwischen `top` und `bottom` = letzter Schritt der bilinearen Interpolation der 4 Dot-Products. Formel: `lerp(a, b, t) = a + t*(b-a)`

Frage 7 — JPEG Reihenfolge (2. Versuch)

Frage: Reihenfolge A) Quantisierung, B) Chroma Subsampling, C) RGB→YCbCr, D) Huffman+RLE, E) DCT?

Deine Antwort: `C → B → E → D → A` (A und D vertauscht)

Feedback:

⚠ Fast — A und D vertauscht. Richtig: **C → B → E → A → D**

- Quantisierung (A) kommt VOR Huffman (D)
- Logik: Erst verlieren (Quantisierung), dann effizient packen (Huffman)
- Huffman ist immer **letzter Schritt**

Frage 8 — applyBrightness

Frage: Schreibe `applyBrightness(arr, c)` .

Deine Antwort:

```
return (arr + c).clip(0, 255).astype(uint8)
```

Feedback:

⚠ Fast richtig — zwei Fehler:

1. `uint8` → muss `np.uint8`
2. `np.int32()` **fehlt** — ohne es: `uint8`-Overflow vor dem `clip`!
`uint8(200) + 100 = 300` → overflow → 44 (falsch, `clip` kommt zu spät)

Korrekt:

```
def applyBrightness(arr, c):  
    return np.int32(arr + c).clip(0, 255).astype(np.uint8)
```

Frage 9 — Container vs. Codec (Aussage beurteilen)

Frage: Welcher Satz ist fachlich falsch?

- A) "WAV ist unkomprimiert"
- B) "WAV enthält PCM-Daten"
- C) "FLAC ist ein verlustfreier Codec"

Deine Antwort: C

Feedback:

X Richtige Antwort: **A**

"WAV ist unkomprimiert" ist ungenau/falsch — WAV ist ein **Container**, kein Codec. WAV kann theoretisch auch komprimierte Daten enthalten. Korrekt: **"WAV enthält üblicherweise PCM-Daten"**.

C ist korrekt: FLAC ist tatsächlich ein verlustfreier Codec.

Frage 10 — Mandelbrot

Frage:

- a) Pixel erreicht `max_iteration = 100` ohne zu divergieren — welche Farbe?
- b) Was repräsentiert `c` geometrisch?

Deine Antwort: a) Schwarz — b) Mandelbrot?

Feedback:

- a) ✓ Schwarz — innerhalb der Menge
- b) $X \rightarrow c$ ist eine **komplexe Zahl** die einem **Punkt auf der komplexen Zahlenebene** entspricht:
X-Achse = Realteil, Y-Achse = Imaginärteil. Jeder Pixel wird auf ein $c = \text{real} + \text{imag} \cdot i$ gemappt.

ZUSAMMENFASSUNG — SCHWACHSTELLEN

Thema	Status	Was merken
Dateigröße-Formel	⚠	$\times (\text{BitDepth}/8)$ — $\div 8$, nicht $\div 2$
Samplerate vs. Bittiefe	X	Samplerate $\rightarrow$ Frequenzbereich (Nyquist); Bittiefe $\rightarrow$ Quantisierungsfehler/Dynamik
BGR-Reihenfolge	✓ R2	B=0, G=1, R=2
<code>cv.COLOR_BGR2GRAY</code>	⚠	Auswendig lernen
YCbCr Koeffizienten	⚠	0.299 / 0.587 / 0.114 (nicht 0.578, nicht 0.141)
JPEG Reihenfolge	⚠	C$\rightarrow$B$\rightarrow$E$\rightarrow$A$\rightarrow$D — Quantisierung VOR Huffman
NumPy <code>np.int32()</code>	⚠	Immer vor Addition/Multiplikation wegen uint8-Overflow
NumPy vektorisiert	⚠	Kein for-loop — direkt auf Array operieren
Container vs. Codec	⚠	WAV $\neq$ unkomprimiert; WAV = Container mit PCM
Perlin Fade	✓ R2	$6t^5 - 15t^4 + 10t^3$
Mandelbrot	✓	$z = z^2 + c$, schwarz = innerhalb, Farbe = Escape-Geschwindigkeit

SCHNELL-REFERENZ — WICHTIGSTE FORMELN

```
# Dateigröße WAV
groesse = dauer * samplerate * kanaele * (bittiefe // 8)

# Sinuston
t = np.linspace(0, dauer, int(sr * dauer), endpoint=False)
signal = 0.5 * np.sin(2 * np.pi * frequenz * t)
audio = (signal * 32767).astype(np.int16)

# YCbCr
Y = 0.299 * R + 0.587 * G + 0.114 * B
Cb = (B - Y) * 0.564
Cr = (R - Y) * 0.713

# Bildtransformationen
def applyBrightness(arr, c): return np.int32(arr + c).clip(0, 255).astype(np.uint8)
def applyContrast(arr, a):  return np.int32(arr * a).clip(0, 255).astype(np.uint8)
def applyNegative(arr):     return (255 - arr).astype(np.uint8)
def applyGamma(arr, g):    return np.int32(255 * (arr/255)**g).clip(0, 255).astype(np.uint8)

# Perlin Fade
def fade(t): return 6*t**5 - 15*t**4 + 10*t**3

# Mandelbrot
c = complex(real, imag)
z = 0
for i in range(max_iteration):
    if abs(z) <= 2:
        z = z**2 + c
    else:
        break
```

JPEG Pipeline: C → B → E → A → D

(RGB→YCbCr → Chroma Sub → DCT → **Quantisierung** → Huffman+RLE)

OpenCV BGR: blue[:, :, 0] , green[:, :, 1] , red[:, :, 2]